

Design and Implementation of a Campus-Centric Carpooling System

Prithviraj Ghosh (230794) Mainak Sarkar (230619) Khushi Ranawat (230562) Mahir Jain (230618) Chaitanya Nitawe (230315)

1. Project Title & Description

Campus Ride-Pooling and Split-Fare System. Students travelling from campus to nearby transit hubs (railway station, airport, etc.) currently book cabs individually and pay full fare, even when several people are headed to the same place at the same time. This project is a web platform that matches such riders on overlapping departure windows, allocates them seats in a shared vehicle, splits the fare, and coordinates pickup, with a governed AI agent assisting in the workflow.

2. User Roles

Rider / Ride Owner: Creates, modifies, or cancels rides; requests to join existing rides; accepts/rejects co-rider join requests when acting as ride owner; pays their fare share; rates co-riders.

Operations Admin: Manages registered vehicles and vehicle capacity; configures destination hubs and fare bands; handles SOS incidents; reviews the AI agent's action log; issues warnings or suspends accounts.

Authentication: IITK email-based authentication for riders, with separate role-based authorization for Operations Admin access.

3. Core Transactional Workflow

Following IITK email authentication, new users complete a quick setup, selecting from a pre-fixed set of tags/interests (e.g., academic tracks, hobbies, or “quiet ride”); this data directly feeds the AI matchmaking agent.

A user initiates a ride with a departure time and preferred vehicle (cab or auto), or requests to join an existing one from AI-generated suggestions based on similar start/destination → co-riders join pending ride-owner approval → the fare split is decided (re-computed if a new rider joins dynamically) → the trip executes and completes.

Once the dynamic fare split locks in, a temporary real-time in-app chat automatically opens for the active pool to coordinate the exact physical meetup spot at the campus gates. Post-ride, users settle the fare and may file detailed text complaints or rate co-riders; these reports are routed immediately to the AI Complaint Management system for severity scoring and potential disciplinary action.

4. Shared, Contended Resource & Concurrency Challenge

(i) **Vehicle Capacity – Stale Availability:** The core concurrency challenge is ensuring riders do not join a ride based on outdated seat availability. E.g., a rider may see one seat available and attempt to join while the ride owner concurrently accepts a different pending rider for that same seat; without proper concurrency control the first request could still be accepted on stale data, causing overbooking. The system must therefore *revalidate seat availability and update the ride atomically* when a rider is accepted.

(ii) **Time of Departure – Dynamic Itinerary Drift:** Adding a new rider introduces pickup detours that shift departure and arrival times for everyone. If multiple users apply concurrently against an outdated departure time,

the global ETA can drift mid-transaction, causing missed pickup windows and itinerary conflicts.

5. AI Agent

(i) **Pool Car Recommendation System:** An AI agent, activated via an “Ask AI to Suggest” button, ranks pool cars for searching riders, generating personalized pros/cons based on pricing, route efficiency, shared-profile interests, user reviews, and past complaints. For incoming join requests, the agent gives the ride owner a compatibility summary based on relevant ride preferences and logistics. An interactive AI chatbot additionally lets users filter rides, inspect co-rider details, and manage pool preferences via natural-language queries.

(ii) **Complaint Management System:** The AI summarizes complaints, assigns a severity score, and categorizes them with behavioral tags (e.g., safety, tardiness). By recognizing recurring patterns in a user's history, it proactively generates enforcement triggers – warnings or suggested suspensions – routed to an Admin Dashboard for quick, one-click human approval.

6. Proposed Technology Stack

- TypeScript
- Node.js (backend)
- Firestore / PostgreSQL
- LLM API (free tier)
- React.js / Next.js
- Socket.IO (real-time communication)
- GCP – Cloud Run, Cloud SQL, Cloud Scheduler, Cloud Logging

7. Initial Risks & Technical Uncertainties

- Concurrency correctness under simultaneous seat and ownership actions
- LLM hallucination in recommendation and complaint scoring
- API and cloud infrastructure cost management
- Designing a robust and fair complaint-management pipeline

8. Jira / GitHub

GitHub repository:

https://github.com/mainak9093/Campus_Ridepooling_Automation – Ride-pooling and split-fare platform for campus travel: matches riders on overlapping departure windows, allocates shared vehicle seats under concurrency control, and splits fares.

Jira project:

<https://cs455project.atlassian.net/jira/software/projects/SCRUM/boards/1>

Instructor and TAs will be added to both the GitHub repository and the Jira project prior to submission.